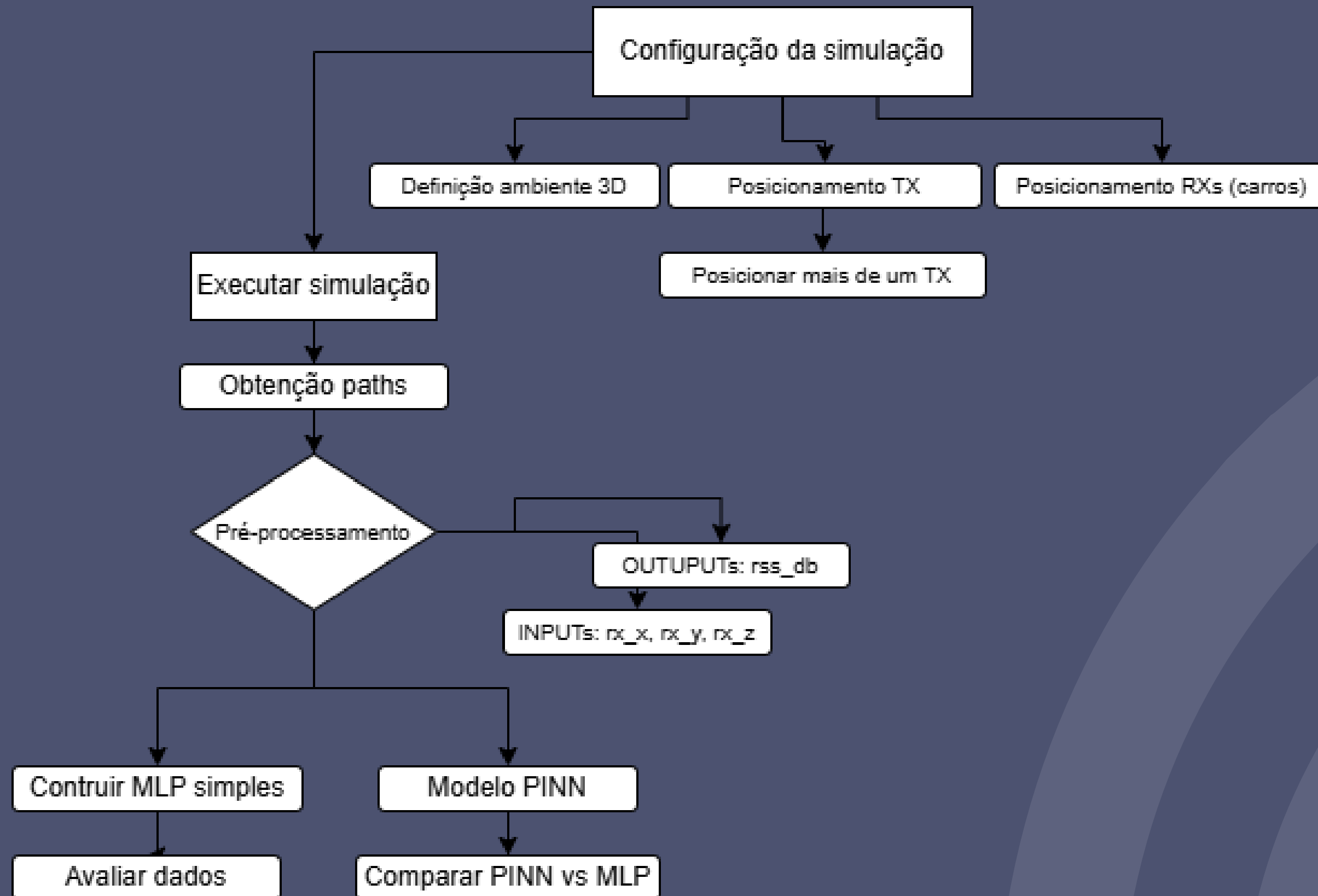


Modelagem de Canal Sem Fio e PINNs

Fluxo de trabalho

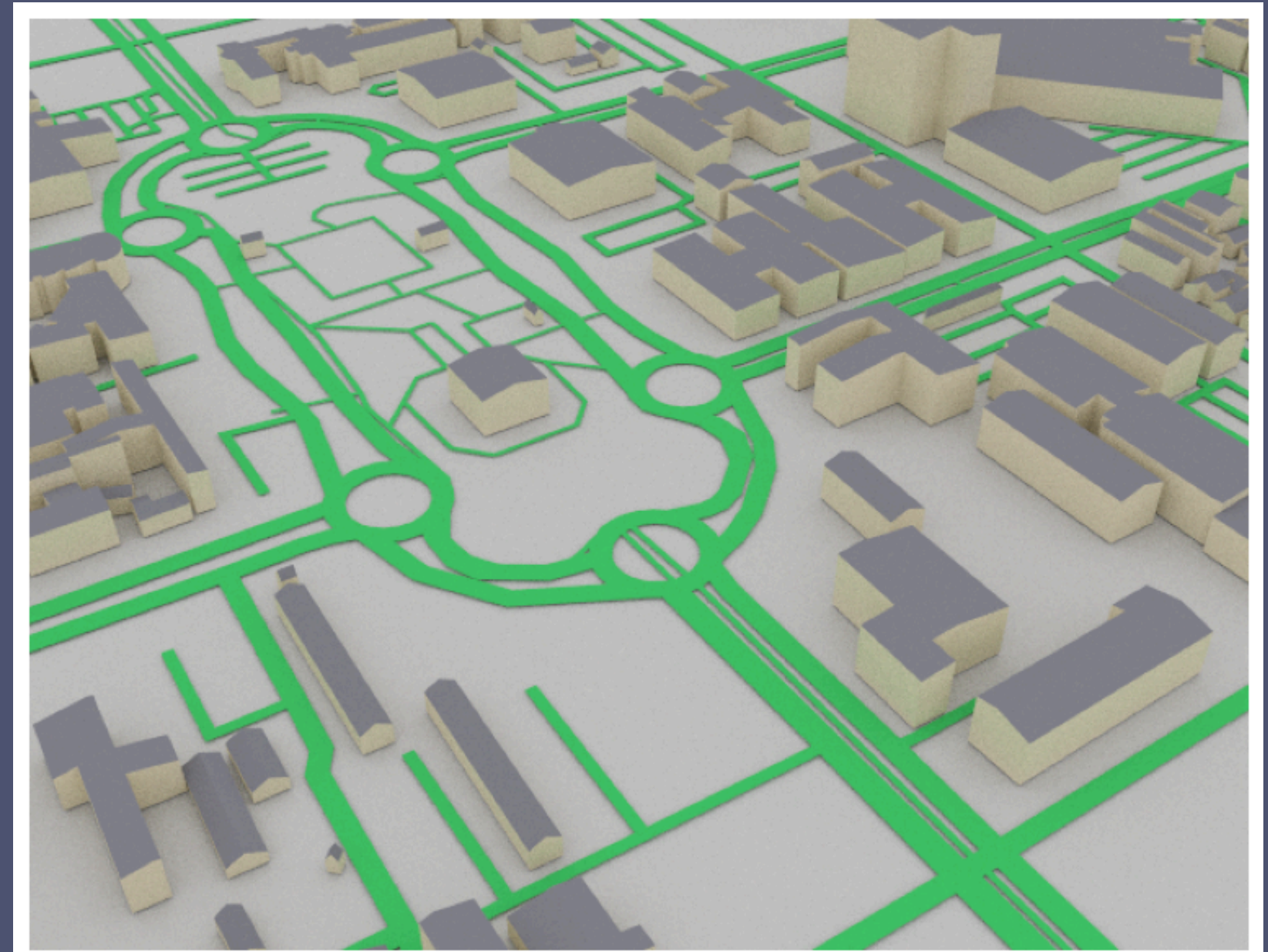


Extraí um ambiente 3D e configurei a cena com materiais reais

Utilizei os materiais do seguinte modo,

- Onde prédios:
 - Telhado: metal
 - Paredes: marble
- Plano: concrete
- E ruas utilizei de um material com permissividade relativa semelhante ao asfalto: plasteboard

Todos presentes na documentação do Sionna de ITURadioMaterials para simplificar de início o projeto.

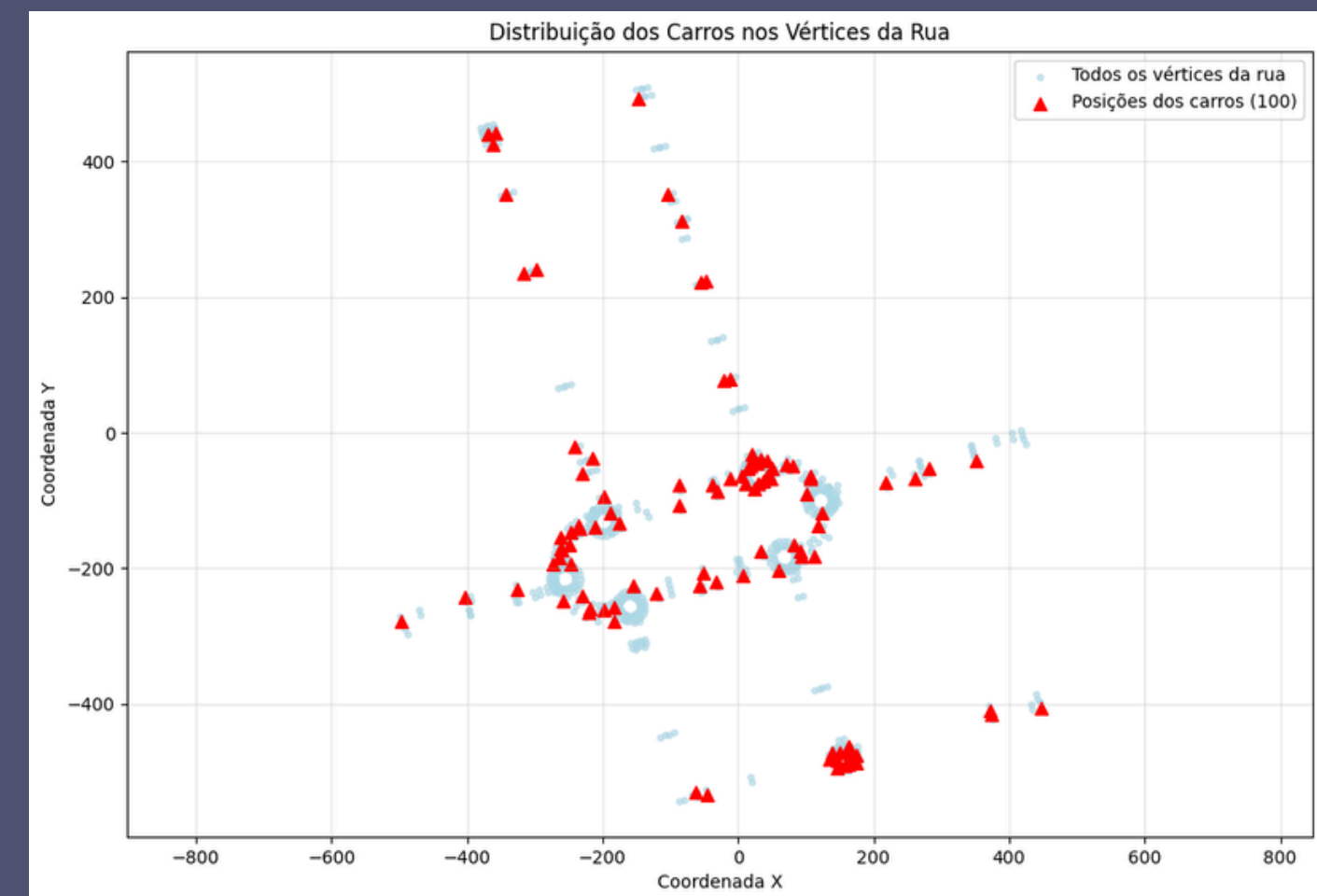
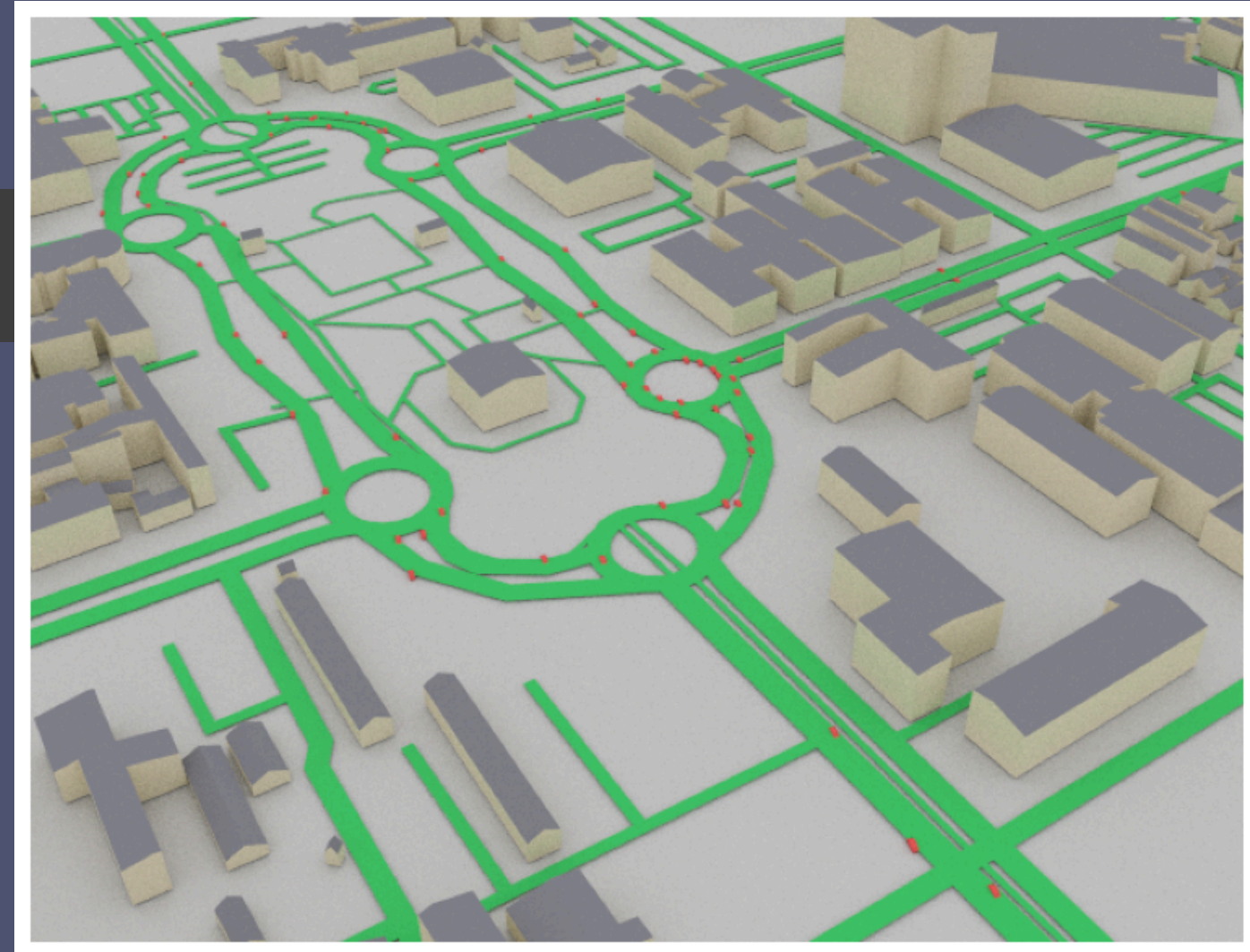


Configuração dos receptores RX

Para a simulação, inicialmente foram posicionados 100 carros nas ruas.

Foram definidos por padrão do Sionna, definidos na cor vermelha e com material ITU “metal”.

Consequentemente cada carro foi associado à um receptor na cena.



Configuração transmissor TX

Delimitado um transmissor na cena em uma posição média em relação aos receptores, no caso da cena na parte inferior da praça universitária.

```
# Add a transmitter on top of a building  
scene.remove("tx")  
scene.add(Transmitter("tx", position=[21.18, -132.4, 18.76], display_radius=2))
```

Com toda a configuração realizada, é realizada a simulação e retornado os “paths” de cada transmissor e receptor, limitado a 5 reflexões de sinal.



Impressão de parâmetros da simulação

```
print(paths.a)
print(paths.tau)
```

Python

```
([[[[[1.80829e-06, 6.24338e-06, -5.94759e-07, -1.33103e-05, 5.62362e-06, 5.74054e-05, 5.75361e-05, -5.80247e-06, 1.5988e-06, 6.04037e-05, 5.07981e-05, 6.18624e-06, -1.47064e-05, 0, 0, 0, 0, 0, 0, 0]]]],
[[[7.40503e-06, 6.47642e-06, 4.83164e-05, -3.28929e-06, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]],
[[[-5.05389e-06, -1.83592e-06, -1.84089e-05, -5.22533e-06, 6.42125e-06, 1.24207e-05, -1.67222e-05, -4.61709e-06, -1.65932e-06, -2.72221e-06, 3.00519e-06, 5.91491e-06, 3.19478e-05, 0, 0, 0, 0, 0, 0, 0]]]],
.. 94 skipped ..,
[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]],
[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]],
[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]], [[[[[1.31149e-06, -3.61947e-06, 3.33496e-07, -4.3612e-06, -3.2533e-06, -9.32446e-06, -8.6629e-06, -5.11085e-06, 1.17239e-06, -8.92149e-06, -8.23158e-06, -3.62765e-06, -2.20211e-06, -2.01125e-06, 4.92506e-06, -7.75995e-06, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]],
[[[1.39313e-06, -2.83117e-06, 3.12626e-06, -1.88224e-06, 2.82968e-06, -3.65997e-06, 2.80965e-06, 1.30276e-06, -2.56443e-06, -1.22717e-06, -6.4429e-06, -8.84609e-07, -3.00341e-05, 0, 0, 0, 0, 0, 0, 0]]]],
.. 94 skipped ..,
[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]],
[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]],
[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]]])
[[[3.6889e-07, 3.04226e-07, 4.32431e-07, 3.02599e-07, 3.04252e-07, 2.98414e-07, 2.98443e-07, 3.64203e-07, 3.68928e-07, 2.98406e-07, 2.98449e-07, 3.04192e-07, 3.02552e-07, -1, -1, -1, -1, -1, -1, -1]],
[[3.5395e-07, 3.53997e-07, 3.49022e-07, 4.20858e-07, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]],
[[4.64021e-07, 6.3242e-07, 4.40468e-07, 5.54773e-07, 5.38466e-07, 4.60284e-07, 4.40483e-07, 4.64033e-07, 6.3243e-07, 5.41674e-07, 3.52375e-07, 4.44399e-07, 3.47424e-07, -1, -1, -1, -1, -1, -1, -1]],
.. 94 skipped ..,
[[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]],
[[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]],
[[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]]]
```

Obtém-se da simulação os parâmetros para realizar os cálculos de input da nossa rede, sendo:

- Paths.a a atenuação complexa, onde os valores preenchidos em 0 são os caminhos os quais não houve chegada de sinal
- Paths.tau o atraso de propagação em segundos, com -1 correspondendo a valores inválidos, para caminhos que não existem em determinados receptores
- Nessa simulação não foram impressos os valores de paths.theta_r(ângulo de chegada) e paths.k(vetor de onda)

Tratamento de dados para a rede

Para o treinamento do modelo, há a necessidade do tratamento dos dados para realização dos cálculos.

- Necessário mapear o vetor de atenuações e atrasos, que no resultado de atenuações temos, **(num_rx, num_rx_ant, num_tx, num_tx_ant, max_paths) (100, 1, 1, 1, 20)**, onde:
 - num_rx = 100
 - num_rx_ant = 1
 - num_tx = 1
 - num_tx_ant = 1
 - max_paths = 20
 - Obs: No tratamento de dados há a necessidade de comprimir esse vetor, realizando um “squeeze” e retirando valores unitários.
- E de atrasos, **(num_tx, max_paths) (1, 20)**
 - num_tx = 1
 - max_paths = 20

Valores tratados, realiza-se a conversão para dBm:

Com os arrays multidimensionais ajustados, calculamos os valores da intensidade dos sinais recebidos (RSS), somando a potência de cada caminho individual:

$$P_{\text{total}} = \sum |a_i|^2$$

Valores de RSS para cada Rx:

```
# 2. Agora, continue com o cálculo do RSS usando o tensor 'squeezed'
# que tem o shape correto (100, 20).

power_per_path = tf.square(tf.abs(atenuações_squeezed))

# Somar as potências ao longo do eixo dos caminhos (axis=1)
# Agora isso funcionará perfeitamente.
total_power_per_rx = tf.reduce_sum(power_per_path, axis=1)

# Converter para dB
epsilon = 1e-10
rss_db_per_rx = 10.0 * tf.math.log(total_power_per_rx + epsilon) / tf.math.log(10.0)

# Imprimir o resultado para verificar
print("\nRSS em dB para cada receptor (para todos os valores):\n", rss_db_per_rx.numpy()[:100])
```

```
RSS em dB para cada receptor (para todos os valores):
[ -78.70407  -85.94812  -86.847855 -84.430305 -79.701866 -80.38556
  -100.      -100.      -100.      -100.      -100.      -99.41119
  -100.      -100.      -100.      -100.      -100.      -93.839455
  -93.84193  -93.440735 -95.35021  -99.87275  -99.24916  -100.
  -100.      -95.78017  -95.82884  -83.9225   -82.47811  -81.64787
  -82.05879  -83.331024 -82.75574  -99.229645 -95.33542  -94.15325
  -84.521935 -85.83892  -82.50171  -91.60391  -85.90017  -85.93651
  -93.04853  -91.78932  -92.8094   -94.82554  -100.      -100.
  -100.      -96.73133  -100.      -100.      -100.      -100.
  -95.35047  -93.12137  -93.19826  -93.75152  -92.82754  -93.06062
  -100.      -100.      -100.      -74.77348  -82.41044  -97.63961
  -88.30433  -83.3131   -81.23942  -82.130875 -92.65389  -99.99852
  -87.286354 -82.733604 -77.7968   -90.789154 -100.      -97.504364
  -90.47225  -90.74405  -99.99872  -91.61717  -100.      -100.
  -85.24716  -81.428024 -98.74815  -98.68921  -95.97017  -96.627594
  -95.93363  -100.      -100.      -100.      -100.      -100.
  -99.99999  -99.99999  -99.99999  -99.99999 ]
```


Obtém-se o dataset para treinamento do modelo

```
dataset_rx_e_potencia.csv
1 rx_x,rx_y,rx_z,rss_db
2 93.2115,-183.1496,3.3,-78.70407
3 112.1458,-181.7198,3.3,-85.94812
4 123.1964,-118.233,3.3,-86.847855
5 117.2265,-137.2343,3.3,-84.430305
6 92.5267,-174.3533,3.3,-79.701866
7 82.5075,-166.0128,3.3,-80.38556
8 -147.5113,493.4142,3.3,-100.0
9 -103.4717,351.3156,3.3,-100.0
10 -83.1653,312.4296,3.3,-100.0
11 -55.1756,221.1227,3.3,-100.0
12 -21.4815,76.9961,3.3,-100.0
13 20.4931,-32.7429,3.3,-99.41119
14 -368.2762,439.0886,3.3,-100.0
15 -342.6609,351.515,3.3,-100.0
16 -316.4062,233.9779,3.3,-100.0
17 -240.5545,-20.7564,3.3,-100.0
18 -229.9874,-61.0203,3.3,-100.0
19 -246.2172,-194.8064,3.3,-93.839455
20 -259.499,-172.4816,3.3,-93.84193
21 -248.0115,-164.8701,3.3,-93.440735
22 -234.4481,-140.4615,3.3,-95.35021
23 -211.6732,-139.4956,3.3,-99.87275
24 32.6729,-39.7381,3.3,-99.24916
25 -11.768,79.6273,3.3,-100.0
26 -47.5975,224.265,3.3,-100.0
27 28.7647,-44.6275,3.3,-95.78017
28 42.3561,-41.407,3.3,-95.82884
29 50.378,-53.084,3.3,-83.9225
30 42.4611,-64.667,3.3,-82.47811
```

Temos um dataset composto pelas coordenadas de cada receptor, em x, y, z.

E consequentemente o valor de RSS em dBm para cada receptor.

Modelo básico

Inicialmente é criada uma arquitetura simples da Rede Neural, com 3 neurônios de entrada que esperam os valores de coordenadas, e 1 de saída para prever o valor de RSS.

Treinamento do modelo com 150 épocas e utilizando 20% dos dados de treino para validação de cada época

```
baseline_model = Sequential([
    # Camada de entrada: espera vetores de 3 elementos (rx_x, rx_y, rx_z)
    Dense(64, activation='relu', input_shape=(3,)),

    # Camadas ocultas
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),

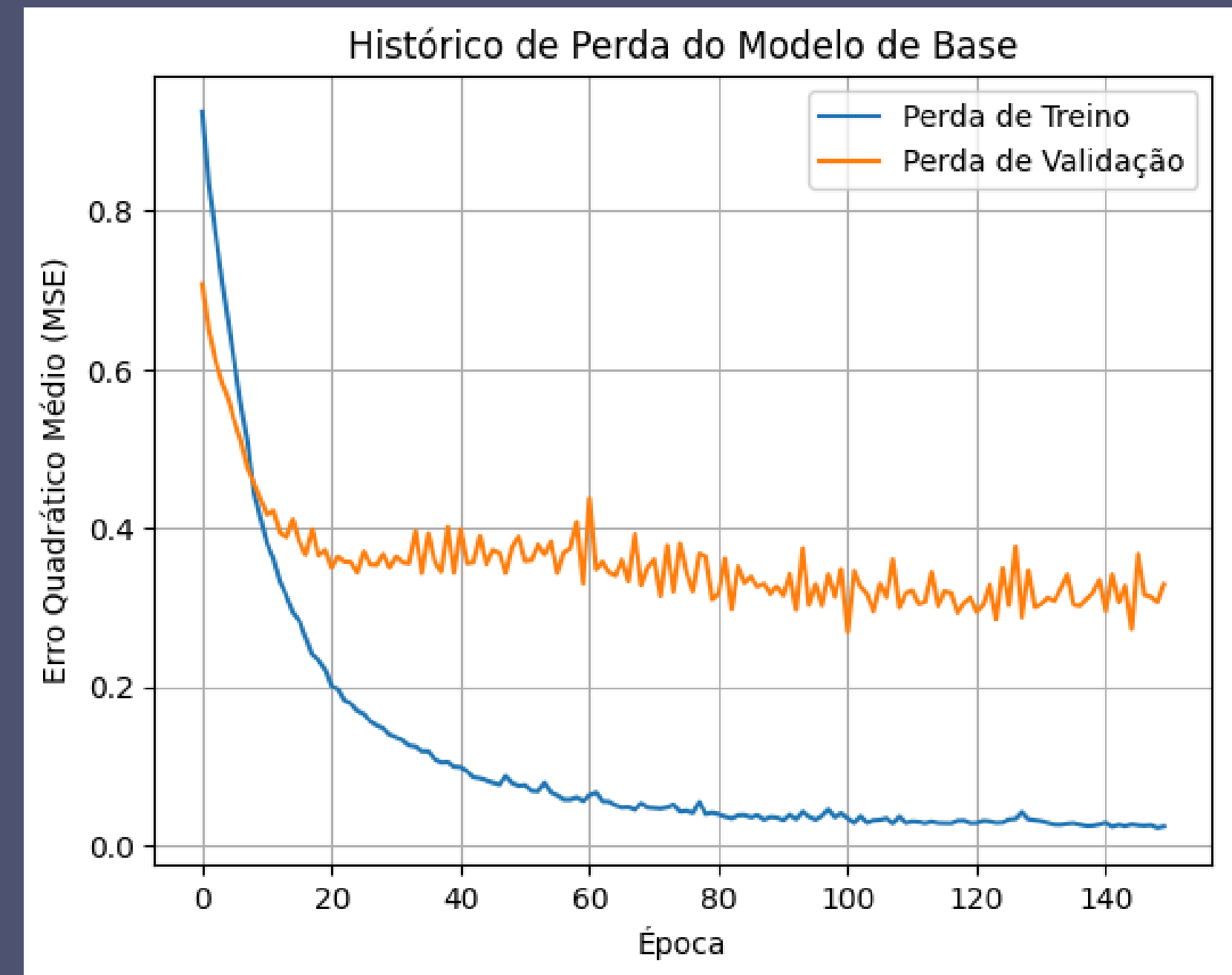
    # Camada de saída: 1 neurônio para prever um único valor (RSS)
    # A ativação 'linear' é o padrão para regressão, então não precisa ser especificada.
    Dense(1)
])
```

```
# Treinamos com os dados normalizados
history = baseline_model.fit(
    X_train_scaled,
    y_train_scaled,
    epochs=150,
    batch_size=8,           # Número de amostras por atualização de gradiente
    validation_split=0.2,   # 20% dos dados de treino para validação a cada época
    verbose=0
)
```

Loss e Validation Loss

Vemos que o modelo funciona dado a curva azul suave, o modelo aprende a se ajustar aos dados, minimizando os erros.

A curva laranja, mesmo que ruidosa, demonstra que o modelo estabiliza e não há overfitting óbvio.

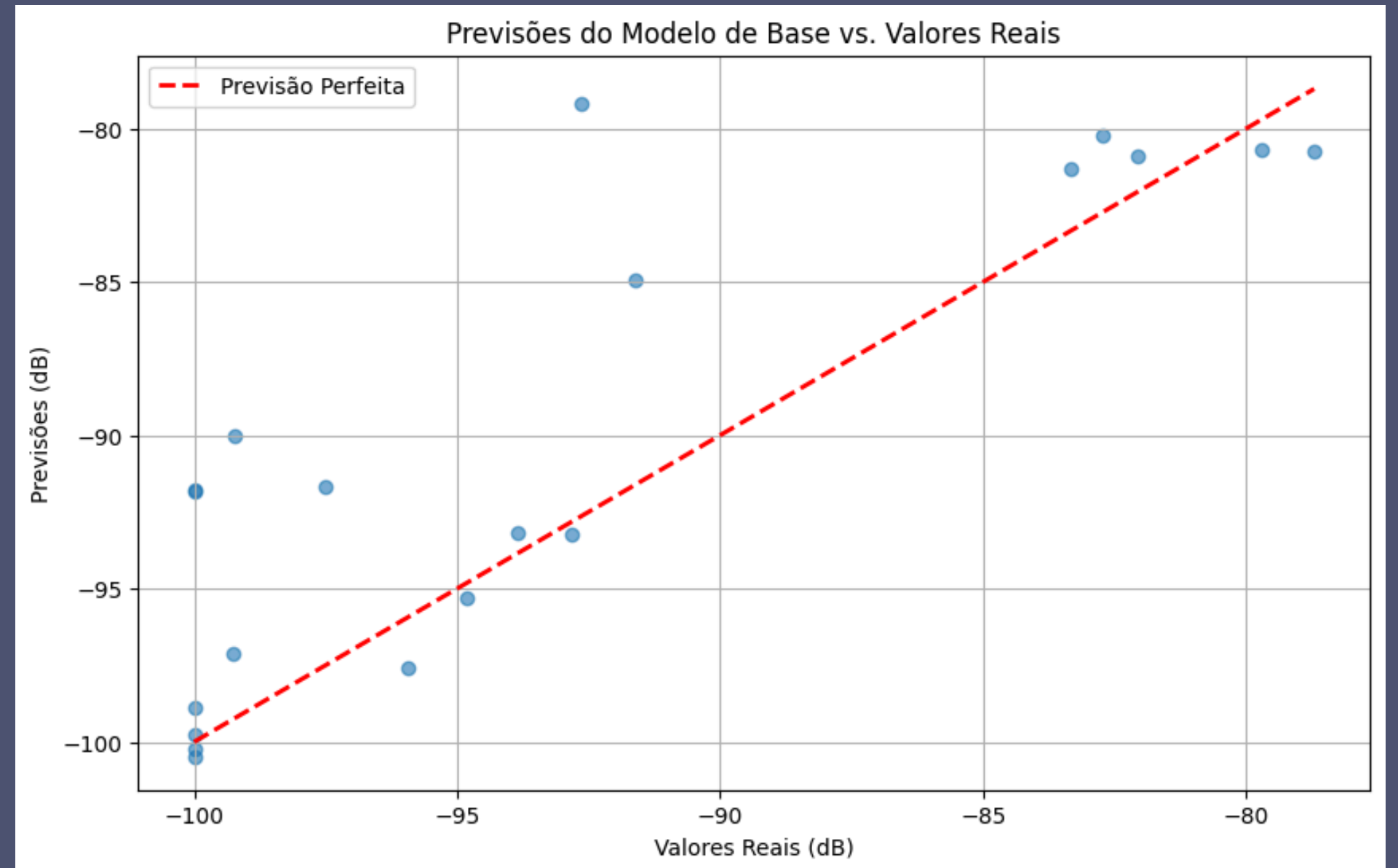


Previsões do modelo base e Valores reais

Performance do Modelo de Base:

O modelo fornece um erro médio absoluto no conjunto de ± 3.39 dB

Um resultado relativamente realista, dado a previsão da potência do sinal com essa precisão em um ambiente complexo.



Arquitetura PINN

O intuito é a implementação de um modelo PINN, a fim de generalizar o modelos para diversas situações, dado que MLPs tendem a falhar quando não há um treino específico.

Para comparar resultados, implementei um modelo PINN de arquitetura similar, mas ativação tanh e perda composta:

$$\mathcal{L} = \underbrace{\omega_d \frac{1}{N_d} \sum (u - u_{\text{obs}})^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\omega_p \frac{1}{N_c} \sum (\nabla^2 u)^2}_{\mathcal{L}_{\text{PDE}}}$$

Previsões do modelo PINN e Valores reais

Esse modelo PINN constatou-se ineficiente:

O modelo fornece um erro médio absoluto no conjunto de ± 3.84 dB, maior que o de 3.39 db do modelo base.

O desempenho do modelo obtido é insatisfatório, possivelmente, pela falta de um volume adequado de dados. O que não permite treinar o modelo de forma eficaz.

